

JavaScript: The New Parts

Here's another article, the ninth in the series on JavaScript. It is about ES6 features that make programs readable and maintainable. Readers joining us now would do well to look up the previous eight parts if they wish to, to get a good grounding in ES6.

In this article, we look at features such as iterables, iterators and Symbols. We will also bring together different ES6 features in one example. The article builds on previously covered features such as collections and *for..of* loop.

Iterables and iterators are protocols. These are available to process elements in a collection, one at a time. If you have a Java or a Python background, you probably are already familiar with these. A protocol is synonymous with an interface in Java. An object obeys the protocol, when it implements specific functions. To make an object iterable, it must implement the *Symbol.iterator* method. For an iterator object, it must implement method *next()*. To use built-in collections like arrays, you do not need to understand iterables and iterators. But, for creating custom iterators and iterables for the objects you have created, you must know about them. By doing that, you make the object walkthrough in the new looping structure *for..of*. Till ES6, a programmer had to implement an iterator on a collection in a cumbersome way. A good iterator design pattern hides the implementation from the algorithm.

An *iterable* makes an object loop through the *for..of* control structure. An *iterator* object knows the logic of how to walk through the collection and remembers the current position.

Iterables

You are already familiar with iterables. Strings and arrays are built-in language iterables. Iterable objects are collections that implement the method [*Symbol.iterator*]. This method makes the collection iterable in the *for..of* loop. This new looping mechanism is specifically for collections. ES6 introduced collections like Maps, Sets, WeakMaps and WeakSets, which are all iterables that can be used with the *for..of* loop.

It is important to understand iterables, as many new constructs and methods expect iterables. For example, in the last issue, we learnt about Promises. The *promise.all()* method takes an iterable object as a parameter. Similarly, the spread operator can be used on objects that are iterable.

An example of an built-in iterable array is shown below:

```
let arr = ['one', 'two', 'three'];
for (let a of arr) {
  console.log(a);
}
```

Example 1: The *for..of* loop for an array

Symbol

Till ES6, there were only six types. Every value in JavaScript was of one of the types - undefined, null, Boolean, Number, String and Object. A new type called Symbol has been added to the JavaScript language. A Symbol is a unique value. It is used to create an object property.

There is a good reason why a Symbol is used for iterables. Due to a lack of proper iteration convenience in JavaScript prior to the arrival of ES6, programmers implemented iterators with custom methods. The purpose of uniqueness in object property, using Symbol, is to avoid any clash with existing methods.

In the browser console, you can check the

following code:

```
> typeof Symbol()
"Symbol"
```

A new looping structure

It is important to understand the difference between *for..in* and *for..of*. Note that *for..in* takes an object and iterates through the keys of the object. Whereas, *for..of* takes an *iterable* object. In real world examples, DOM navigation methods are going to be iterable objects that return NodeList.

```
for (variable in object) {
  //statement
}
for (variable of iterable) {
  //statement
}
```

An iterable is easily mistaken for an array. For example, a string in JavaScript is an iterable and hence can be used with a *for..of* loop.

```
let str = "hello";
for (const s of str) {
  console.log(s);
}
```

Looping over an ES6 new data structure set is shown below:

```
let set1 = new Set([11, 22, 33, 11, 33]);
for (const s of set1) {
  console.log(s);
}
```

